Assignment 9

Week 10

6733172621 Patthadon Phengpinij
Collaborators. Claude (for LaTeX styling and grammar checking)

Problem 1. The General Form of XOR Quantum Oracle

Assume $x = 5$, $f(5) = 3$, $n = 3$, and $m = 3$, find the corresponding input and output basis vectors of the XOR quantum oracle corresponding to $f(x)$.

Solution

Setting. With $n = 3$ and $m = 3$, the XOR oracle acts on $n + m = 6$ qubits. The query $|x\rangle_3$ is the 3 MSB and the auxiliary register $|y\rangle_3$ is the 3 LSB. As in the lecture example, the auxiliary qubits start in $|0\rangle_3$.

Input basis vector.

$$|x\rangle_3 |y\rangle_3 = |5\rangle_3 |0\rangle_3 = |101\rangle |000\rangle = |101000\rangle \quad (= |40\rangle_6).$$

Output basis vector. U_f keeps the 3 MSB and XORs $f(5) = 3 = (011)_2$ into the 3 LSB:

$$\begin{aligned} U_f |5\rangle_3 |0\rangle_3 &= |5\rangle_3 |0 \oplus f(5)\rangle_3 = |5\rangle_3 |0 \oplus 3\rangle_3 \\ &= |101\rangle |000 \oplus 011\rangle = |101\rangle |011\rangle \\ &= |101011\rangle \quad (= |43\rangle_6). \end{aligned}$$

For a general auxiliary input the output is $|101\rangle |y \oplus 011\rangle$. Feeding $|101011\rangle$ back in gives $|101\rangle |011 \oplus 011\rangle = |101000\rangle$, the input, as expected from $U_f U_f = I$.

Problem 2. Unitarity of XOR Quantum Oracle

Prove that the XOR quantum oracle is unitary. Prove it using the same methodology we used for the phase quantum oracle.

Solution

Method. For the phase oracle we showed that the transformed basis vectors are still orthonormal, so U_f is unitary. We do the same here. Take two basis vectors of the $(n+m)$-qubit space, $|x\rangle_n |y\rangle_m$ and $|x'\rangle_n |y'\rangle_m$. Their inner product is

$$\left(\langle x|_n \langle y|_m\right) \left(|x'\rangle_n |y'\rangle_m\right) = \langle x|x'\rangle \langle y|y'\rangle = \delta_{xx'} \delta_{yy'}.$$

After the oracle they become $|x\rangle_n |y \oplus f(x)\rangle_m$ and $|x'\rangle_n |y' \oplus f(x')\rangle_m$. These are again basis vectors, so their coefficients are real and no complex conjugate appears. The new inner product is

$$\langle x|x'\rangle \ \langle y \oplus f(x)|y' \oplus f(x')\rangle .$$

- **If $x \neq x'$:** $\langle x|x'\rangle = 0$, so the inner product is still 0, whatever $f(x)$ and $f(x')$ are.
- **If $x = x'$:** then $f(x) = f(x') = c$, one fixed m-bit string. Bit-wise XOR with a fixed c is one-to-one, because $y \oplus c = y' \oplus c \Rightarrow y = y'$ (XOR both sides with c). So $\langle y \oplus c|y' \oplus c\rangle = \delta_{yy'}$, and the inner product is 1 if $y = y'$ and 0 otherwise.

In both cases $\langle x|x'\rangle \langle y \oplus f(x)|y' \oplus f(x')\rangle = \delta_{xx'}\delta_{yy'}$, the same as before. The 2^{n+m} basis vectors are mapped to 2^{n+m} orthonormal vectors, so the columns of U_f are orthonormal, i.e. $U_f^\dagger U_f = I^{\otimes(n+m)}$. By linearity the inner product of *any* two states is preserved too. Hence the XOR oracle is unitary.

> **Remark.** The proof never needs f to be one-to-one. Several inputs x may give the same $f(x)$, and the XOR oracle is still unitary because it keeps $|x\rangle$ in the output. That is why the auxiliary register is added. The notebook (**Appendix 1**) builds U_f for five random, non-injective $f : \{0,1\}^3 \to \{0,1\}^3$ and confirms $U_f^\dagger U_f = I$ and $U_f U_f = I$ (64×64) for each.

Problem 3. Construction of SWAP-gate

Construct the matrix for SWAP gate using the following equation:

$$U = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \langle j| U |i\rangle |j\rangle \langle i|$$

Solution

Definition. SWAP exchanges the two qubits: $\text{SWAP}|q_1 q_0\rangle = |q_0 q_1\rangle$. On the basis vectors,

$$\text{SWAP}|00\rangle = |00\rangle, \quad \text{SWAP}|01\rangle = |10\rangle, \quad \text{SWAP}|10\rangle = |01\rangle, \quad \text{SWAP}|11\rangle = |11\rangle.$$

Matrix elements. With $N = 4$, there are 16 terms $\langle j|\,\text{SWAP}\,|i\rangle\,|j\rangle\,\langle i|$. By orthonormality, only four are non-zero:

$$\langle 00|\,\text{SWAP}\,|00\rangle = \langle 00|00\rangle = 1, \qquad \langle 10|\,\text{SWAP}\,|01\rangle = \langle 10|10\rangle = 1,$$
$$\langle 01|\,\text{SWAP}\,|10\rangle = \langle 01|01\rangle = 1, \qquad \langle 11|\,\text{SWAP}\,|11\rangle = \langle 11|11\rangle = 1.$$

Sum of outer products.

$$\begin{aligned}
\text{SWAP} &= |00\rangle\langle 00| + |10\rangle\langle 01| + |01\rangle\langle 10| + |11\rangle\langle 11| \\
&= \begin{pmatrix}1\\0\\0\\0\end{pmatrix}\begin{pmatrix}1 & 0 & 0 & 0\end{pmatrix} + \begin{pmatrix}0\\0\\1\\0\end{pmatrix}\begin{pmatrix}0 & 1 & 0 & 0\end{pmatrix} \\
&\quad + \begin{pmatrix}0\\1\\0\\0\end{pmatrix}\begin{pmatrix}0 & 0 & 1 & 0\end{pmatrix} + \begin{pmatrix}0\\0\\0\\1\end{pmatrix}\begin{pmatrix}0 & 0 & 0 & 1\end{pmatrix} \\
&= \begin{pmatrix}1&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&0\end{pmatrix} + \begin{pmatrix}0&0&0&0\\0&0&0&0\\0&1&0&0\\0&0&0&0\end{pmatrix} + \begin{pmatrix}0&0&0&0\\0&0&1&0\\0&0&0&0\\0&0&0&0\end{pmatrix} + \begin{pmatrix}0&0&0&0\\0&0&0&0\\0&0&0&0\\0&0&0&1\end{pmatrix} \\
&= \begin{pmatrix}1&0&0&0\\0&0&1&0\\0&1&0&0\\0&0&0&1\end{pmatrix}.
\end{aligned}$$

The notebook (**Appendix 1**) builds the same sum numerically and checks it against Qiskit's `SwapGate`.

Problem 4. Deutsch Algorithm

Find the quantum oracle for f_C and f_D in the following equation:

$$f_A(0) = 0, \text{ and } f_A(1) = 0$$
$$f_B(0) = 0, \text{ and } f_B(1) = 1$$
$$f_C(0) = 1, \text{ and } f_C(1) = 0$$
$$f_D(0) = 1, \text{ and } f_D(1) = 1$$

Implement them on IBM-Q.

Solution

Setting. In the Deutsch algorithm, $n = m = 1$: the query x is the MSB q_1, the auxiliary y is the LSB q_0, and $U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$. Each oracle is built with $U = \sum_{i,j} \langle j|U|i\rangle|j\rangle\langle i|$, as was done for f_B in the lecture.

Oracle for f_C ($f_C(0) = 1$, $f_C(1) = 0$). The basis vectors transform as

$$U_f |0\rangle |0\rangle = |0\rangle |0 \oplus 1\rangle = |0\rangle |1\rangle , \qquad U_f |0\rangle |1\rangle = |0\rangle |1 \oplus 1\rangle = |0\rangle |0\rangle ,$$
$$U_f |1\rangle |0\rangle = |1\rangle |0 \oplus 0\rangle = |1\rangle |0\rangle , \qquad U_f |1\rangle |1\rangle = |1\rangle |1 \oplus 0\rangle = |1\rangle |1\rangle .$$

The only non-zero elements are $\langle 01| U_f |00\rangle = \langle 00| U_f |01\rangle = \langle 10| U_f |10\rangle = \langle 11| U_f |11\rangle = 1$, so

$$U_{f_C} = |01\rangle \langle 00| + |00\rangle \langle 01| + |10\rangle \langle 10| + |11\rangle \langle 11| = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} .$$

It flips the LSB only when $x = 0$. Since $f_C(x) = 1 \oplus x$, $y \oplus f_C(x) = (y \oplus x) \oplus 1$: a CNOT ($y \to y \oplus x$) followed by X on the LSB. Check:

$$(I \otimes X)\,\mathrm{CNOT} = \begin{pmatrix} X & 0 \\ 0 & X \end{pmatrix} \begin{pmatrix} I & 0 \\ 0 & X \end{pmatrix} = \begin{pmatrix} X & 0 \\ 0 & I \end{pmatrix} = U_{f_C} .$$

Oracle for f_D ($f_D(0) = f_D(1) = 1$). Now every basis vector has its LSB flipped:

$$U_f |0\rangle |0\rangle = |0\rangle |1\rangle , \quad U_f |0\rangle |1\rangle = |0\rangle |0\rangle , \quad U_f |1\rangle |0\rangle = |1\rangle |1\rangle , \quad U_f |1\rangle |1\rangle = |1\rangle |0\rangle ,$$

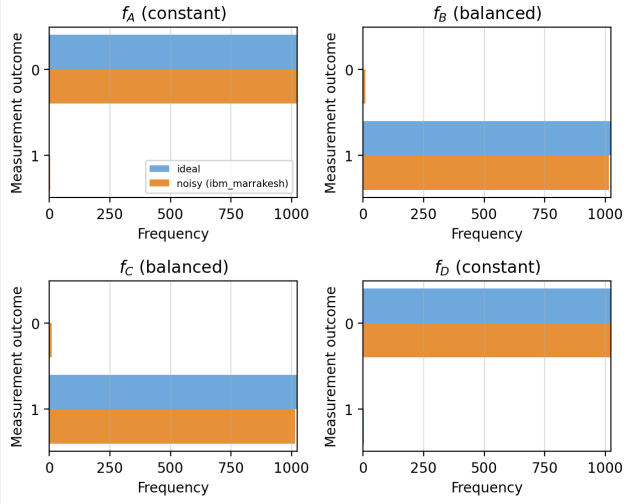
$$U_{f_D} = |01\rangle \langle 00| + |00\rangle \langle 01| + |11\rangle \langle 10| + |10\rangle \langle 11| = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} = I \otimes X,$$

i.e. just an X gate on the auxiliary qubit. For completeness, $U_{f_A} = I$ (no gate) and $U_{f_B} = \mathrm{CNOT}$ (lecture).

Implementation. The four oracles were placed in the Deutsch circuit (auxiliary $|1\rangle \to |-\rangle$, query $|0\rangle \to |+\rangle$, oracle, H on the query, measure the query). The circuits were run with Qiskit's `AerSimulator`, 1024 shots each, instead of on IBM-Q hardware: once ideal, and once with the noise model of the IBM processor `ibm_marrakesh` (Qiskit's `FakeMarrakesh`, a snapshot of the device's calibration data).

```
qc.x(0); qc.h([0, 1])          # auxiliary |->, query |+>
qc.cx(1, 0); qc.x(0)           # U_fC   (U_fD: qc.x(0) only)
qc.h(1); qc.measure(1, 0)      # 0 = constant, 1 = balanced
```

query measured `0` / `1`	f_A (constant)	f_B (balanced)	f_C (balanced)	f_D (constant)
ideal simulator	1024 / 0	0 / 1024	0 / 1024	1024 / 0
`ibm_marrakesh` noise model	1020 / 4	10 / 1014	10 / 1014	1020 / 4



The balanced f_C gives `1` and the constant f_D gives `0`, with one oracle call each. The noisy runs are still more than 98% correct. The query qubit ends in $|f(0) \oplus f(1)\rangle$ up to a global phase, so f_C behaves like f_B and f_D behaves like f_A: the extra X on the auxiliary qubit only adds a global phase -1, because $X|-\rangle = -|-\rangle$. The notebook also checks that each gate circuit has exactly the matrix derived above.

Problem 5. Rotation of $|\Phi\rangle$

Make a table to show the angles between the rotated $|\Phi\rangle$ and $|a\rangle$ and between the rotated $|\Phi\rangle$ and $|a_\perp\rangle$ after applying V, WV, VWV, $WVWV$, …

Solution

Rules. All the vectors lie in the plane of $|a_\perp\rangle$ and $|a\rangle$ (Pic. 29). Let α be the angle of a vector measured from $|a_\perp\rangle$ towards $|a\rangle$. Then its angle to $|a\rangle$ is $90^\circ - \alpha$.

- $|\Phi\rangle$ starts at $\alpha = \theta$, with $\sin\theta = 1/\sqrt{N}$.
- V reflects about $|a_\perp\rangle$: $\alpha \to -\alpha$.
- W reflects about $|\Phi\rangle$, which sits at θ: $\alpha \to 2\theta - \alpha$.

Starting from $(2k+1)\theta$, V gives $-(2k+1)\theta$, and then W gives $2\theta + (2k+1)\theta = (2k+3)\theta$. Each WV therefore adds 2θ.

state	angle to $\|a_\perp\rangle$	angle to $\|a\rangle$	$N=4$: to $\|a_\perp\rangle$	$N=4$: to $\|a\rangle$
$\|\Phi\rangle$	θ	$90^\circ - \theta$	30°	60°
$V\|\Phi\rangle$	$-\theta$	$90^\circ + \theta$	-30°	120°
$WV\|\Phi\rangle$	3θ	$90^\circ - 3\theta$	90°	0°
$VWV\|\Phi\rangle$	-3θ	$90^\circ + 3\theta$	-90°	180°
$WVWV\|\Phi\rangle$	5θ	$90^\circ - 5\theta$	150°	-60°
$VWVWV\|\Phi\rangle$	-5θ	$90^\circ + 5\theta$	-150°	240°
$WVWVWV\|\Phi\rangle$	7θ	$90^\circ - 7\theta$	210°	-120°
⋮	⋮	⋮	⋮	⋮
$(WV)^k\|\Phi\rangle$	$(2k+1)\theta$	$90^\circ - (2k+1)\theta$		
$V(WV)^k\|\Phi\rangle$	$-(2k+1)\theta$	$90^\circ + (2k+1)\theta$		

The angles are signed: a negative angle to $|a\rangle$ means the vector has rotated *past* $|a\rangle$. The unsigned angle between two lines is the absolute value reduced to $[0^\circ, 180^\circ]$. The probability of measuring a after k iterations is $\sin^2((2k+1)\theta)$. It is largest when $(2k+1)\theta \approx 90^\circ$, i.e. $k \approx \frac{\pi}{4\theta} - \frac{1}{2} \approx \frac{\pi}{4}\sqrt{N}$ for large N. For $N=4$, $\theta = 30^\circ$, so a single WV lands exactly on $|a\rangle$, and a second one over-rotates to 150° (**Problem 11**). The notebook (**Appendix 1**) checks every $N=4$ entry against the actual state vectors.

Problem 6. Another Form of V

Show that $V = I - 2|a\rangle\langle a|$. Compare this to W in equation $W = 2|\Phi\rangle\langle\Phi| - I$.

Solution

$V = I - 2|a\rangle\langle a|$**.** Two operators are equal if they act the same way on every basis vector $|x\rangle$. By definition $V|x\rangle = (-1)^{f(x)}|x\rangle$, with $f(a) = 1$ and $f(x) = 0$ for $x \neq a$. On the other side,

$$(I - 2|a\rangle\langle a|)|x\rangle = |x\rangle - 2|a\rangle\langle a|x\rangle = |x\rangle - 2\delta_{ax}|a\rangle = \begin{cases} |x\rangle, & x \neq a \\ -|a\rangle, & x = a \end{cases} = (-1)^{f(x)}|x\rangle = V|x\rangle.$$

The two agree on every basis vector, so $V = I - 2|a\rangle\langle a|$. ■ For example, $n=2$, $a=1$ gives $I - 2|01\rangle\langle 01| = \mathrm{diag}(1,-1,1,1)$, the U_f of the lecture.

Comparison with $W = 2|\Phi\rangle\langle\Phi| - I$.

- **Same form, opposite sign.** $V = -(2|a\rangle\langle a| - I)$ and $W = +(2|\Phi\rangle\langle\Phi| - I)$. Both are built from the projector onto one unit vector ($|a\rangle$ or $|\Phi\rangle$), and they differ by an overall sign.
- **What they reflect about.** W keeps $|\Phi\rangle$ and negates everything orthogonal to it: it reflects about $|\Phi\rangle$. V does the opposite with $|a\rangle$: it negates $|a\rangle$ and keeps everything orthogonal to it, including $|a_\perp\rangle$. In the plane of Pic. 29, that is a reflection about $|a_\perp\rangle$.
- **Global phase.** Since $-(2|a\rangle\langle a| - I)$ and $2|a\rangle\langle a| - I$ differ only by the factor -1, V is, up to a global phase, also "a reflection about $|a\rangle$" of the same form as W. Likewise $W = -(I - 2|\Phi\rangle\langle\Phi|)$.
- **How they are built.** $W = H^{\otimes n}\left(2|0\rangle_n\langle 0|_n - I\right)H^{\otimes n}$, because $H^{\otimes n}|0\rangle_n = |\Phi\rangle$. V needs the oracle: $|a\rangle$ is the unknown answer, so $I - 2|a\rangle\langle a|$ is exactly the information hidden in f.

The notebook (**Appendix 1**) checks $V = I - 2|a\rangle\langle a|$ for all four a with $n = 2$, and $W = H^{\otimes 2}(2|00\rangle\langle 00| - I)H^{\otimes 2} = -(I - 2|\Phi\rangle\langle\Phi|)$.

Problem 7. Numerical Substitution I

If $n = 2$ and $a = 3$, find $|\Phi\rangle$, $|a\rangle$, and $|a_\perp\rangle$.

Solution

With $n = 2$ there are $N = 2^2 = 4$ entries, encoded as $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. The target $a = 3 = (11)_2$.

$$|\Phi\rangle = \frac{1}{\sqrt{2^n}}\sum_{x=0}^{2^n-1}|x\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle) = \frac{1}{2}\begin{pmatrix}1 & 1 & 1 & 1\end{pmatrix}^\top,$$

$$|a\rangle = |3\rangle = |11\rangle = \begin{pmatrix}0 & 0 & 0 & 1\end{pmatrix}^\top,$$

$$|a_\perp\rangle = \frac{1}{\sqrt{2^n - 1}}\sum_{x=0,x\neq a}^{2^n-1}|x\rangle = \frac{1}{\sqrt{3}}(|00\rangle + |01\rangle + |10\rangle) = \frac{1}{\sqrt{3}}\begin{pmatrix}1 & 1 & 1 & 0\end{pmatrix}^\top.$$

Check: $\langle a|a_\perp\rangle = 0$, and $\cos\theta = \langle a_\perp|\Phi\rangle = \frac{1}{\sqrt{3}}\cdot\frac{1}{2}\cdot 3 = \frac{\sqrt{3}}{2}$, so $\theta = \frac{\pi}{6}$, the same as for $a = 1$. θ depends only on N, not on which entry is the target.

Problem 8. Numerical Substitution II

Continue from Problem 7, express $V|\Phi\rangle$ as a linear combination of $|\Phi\rangle$ and $|\Phi_\perp\rangle$.

Solution

$|\Phi_\perp\rangle$. In the plane of Pic. 29, $|\Phi\rangle = \cos\theta|a_\perp\rangle + \sin\theta|a\rangle$, and $|\Phi_\perp\rangle$ is 90° ahead of it, towards $|a\rangle$:

$$|\Phi_\perp\rangle = -\sin\theta|a_\perp\rangle + \cos\theta|a\rangle = -\frac{1}{2}\cdot\frac{1}{\sqrt{3}}(|00\rangle + |01\rangle + |10\rangle) + \frac{\sqrt{3}}{2}|11\rangle$$
$$= \frac{1}{2\sqrt{3}}(-|00\rangle - |01\rangle - |10\rangle + 3|11\rangle).$$

It is normalized, $\frac{1+1+1+9}{12} = 1$, and $\langle\Phi|\Phi_\perp\rangle = \frac{1}{2}\cdot\frac{1}{2\sqrt{3}}(-1-1-1+3) = 0$.
$V|\Phi\rangle$. $V = I - 2|11\rangle\langle 11| = \mathrm{diag}(1,1,1,-1)$ flips the sign of $|a\rangle = |11\rangle$:

$$V|\Phi\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle - |11\rangle).$$

The coefficients follow from inner products with the orthonormal pair $|\Phi\rangle, |\Phi_\perp\rangle$:

$$\langle\Phi|V\Phi\rangle = \tfrac{1}{2}\cdot\tfrac{1}{2}(1+1+1-1) = \tfrac{1}{2},$$

$$\langle\Phi_\perp|V\Phi\rangle = \tfrac{1}{2\sqrt{3}} \cdot \tfrac{1}{2}\,(-1-1-1-3) = -\tfrac{6}{4\sqrt{3}} = -\tfrac{\sqrt{3}}{2}.$$

Therefore

$$\boxed{V\,|\Phi\rangle = \frac{1}{2}\,|\Phi\rangle - \frac{\sqrt{3}}{2}\,|\Phi_\perp\rangle = \cos 2\theta\;|\Phi\rangle - \sin 2\theta\;|\Phi_\perp\rangle}$$

with $2\theta = 60°$. Check: $\tfrac{1}{4}(1,1,1,1) - \tfrac{1}{4}(-1,-1,-1,3) = \tfrac{1}{2}(1,1,1,-1) = V\,|\Phi\rangle$. Geometrically, V moved $|\Phi\rangle$ from $+\theta$ to $-\theta$, i.e. 2θ clockwise from $|\Phi\rangle$, as in Pic. 29. W then reflects about $|\Phi\rangle$ by negating the $|\Phi_\perp\rangle$ coefficient: $WV\,|\Phi\rangle = \tfrac{1}{2}\,|\Phi\rangle + \tfrac{\sqrt{3}}{2}\,|\Phi_\perp\rangle = |11\rangle = |a\rangle$.

Problem 9. Computational Complexity

If algorithm A has a complexity of $O(e^N)$ and algorithm B has a complexity of $O(2N)$. What type of speedup does algorithm B have over algorithm A?

Solution

Exponential speedup. Constant factors do not matter in complexity, so $O(2N) = O(N)$: algorithm B is linear and algorithm A is exponential. Write A's cost as $T_A \sim e^N$. Then $N = \ln T_A$, so B's cost is $T_B \sim N = \ln T_A$: B needs only about the *logarithm* of A's time. This is exactly the lecture's definition of an exponential speedup (from $O(N)$ to $O(\log N)$). Here it is from $O(e^N)$ to $O(\log e^N) = O(N)$.

N	e^N	$2N$	$e^N/2N$
5	148	10	15
10	2.2×10^4	20	1.1×10^3
20	4.9×10^8	40	1.2×10^7
40	2.4×10^{17}	80	2.9×10^{15}

The advantage itself grows exponentially with N. It is not a quadratic speedup: that would be $T_B \sim \sqrt{T_A} = e^{N/2}$, which is what Grover's algorithm gives over classical search ($O(\sqrt{N})$ vs. $O(N)$).

Problem 10. Construction of Oracle

Derive Eq. (1) using Eq. (2) by substituting Eq. (3)

$$U_f = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \tag{1}$$

$$U = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} \langle j|\,U\,|i\rangle\,|j\rangle\,\langle i| \tag{2}$$

$$V\,|x\rangle_n = (-1)^{f(x)}\,|x\rangle_n \tag{3}$$

Solution

Setting. As in the lecture's example, $n = 2$ and $a = 1$, so $f(01) = 1$ and $f(x) = 0$ otherwise. The oracle $U_f = V$ is defined by Eq. (3): $V\,|x\rangle_2 = (-1)^{f(x)}\,|x\rangle_2$.

Matrix elements. Substituting Eq. (3) into $\langle j|\,V\,|i\rangle$,

$$\langle j|\,V\,|i\rangle = \langle j|\,(-1)^{f(i)}\,|i\rangle = (-1)^{f(i)}\,\langle j|i\rangle = (-1)^{f(i)}\delta_{ij},$$

so only the four diagonal terms of Eq. (2) survive:

$$\langle 00|\,V\,|00\rangle = (-1)^{f(0)}\,\langle 00|00\rangle = (-1)^0 = 1, \qquad \langle 01|\,V\,|01\rangle = (-1)^{f(1)}\,\langle 01|01\rangle = (-1)^1 = -1,$$
$$\langle 10|\,V\,|10\rangle = (-1)^{f(2)}\,\langle 10|10\rangle = (-1)^0 = 1, \qquad \langle 11|\,V\,|11\rangle = (-1)^{f(3)}\,\langle 11|11\rangle = (-1)^0 = 1.$$

Sum of outer products.

$$
\begin{aligned}
U_f &= \sum_{i=0}^{3}\sum_{j=0}^{3} \langle j| V |i\rangle |j\rangle \langle i| = |00\rangle\langle 00| - |01\rangle\langle 01| + |10\rangle\langle 10| + |11\rangle\langle 11| \\
&= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix},
\end{aligned}
$$

which is Eq. (1). ■ In general, the phase oracle is always diagonal, $U_f = \sum_x (-1)^{f(x)} |x\rangle\langle x|$, because it never mixes basis vectors; it only changes their sign.

Problem 11. Rotation of $WV|\Phi\rangle$

Perform another WV operation to the wavefunction obtained in the below equation. Is it over-rotated?

$$
\begin{aligned}
&H^{\otimes 2}\left(2|0\rangle_2\langle 0|_2 - I\right) H^{\otimes 2}\left(V|\Phi\rangle\right) \\
&= \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \times \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \times \frac{1}{2}\begin{pmatrix} 1 \\ -1 \\ 1 \\ 1 \end{pmatrix} \\
&= \frac{1}{4}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}\begin{pmatrix} 1 \\ 1 \\ -1 \\ 1 \end{pmatrix} \\
&= \frac{1}{4}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 1 \\ -1 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = |01\rangle = |a\rangle
\end{aligned}
$$

Solution

Starting point. After one WV the state is $|01\rangle = |a\rangle$.

Second V. $V = \mathrm{diag}(1, -1, 1, 1)$ flips the sign of $|01\rangle$:

$$V|01\rangle = -|01\rangle = \begin{pmatrix} 0 & -1 & 0 & 0 \end{pmatrix}^\top.$$

Second W.

$$
\begin{aligned}
&H^{\otimes 2}\left(2|0\rangle_2\langle 0|_2 - I\right) H^{\otimes 2}\left(-|01\rangle\right) \\
&= \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}\frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 0 \\ -1 \\ 0 \\ 0 \end{pmatrix} \\
&= \frac{1}{4}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}\begin{pmatrix} -1 \\ 1 \\ -1 \\ 1 \end{pmatrix} = \frac{1}{4}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}\begin{pmatrix} -1 \\ -1 \\ 1 \\ -1 \end{pmatrix} \\
&= \frac{1}{4}\begin{pmatrix} -2 \\ 2 \\ -2 \\ -2 \end{pmatrix} = \frac{1}{2}\left(-|00\rangle + |01\rangle - |10\rangle - |11\rangle\right).
\end{aligned}
$$

Yes, it is over-rotated. The probability of measuring the answer has dropped from 1 to $\left|\frac{1}{2}\right|^2 = \frac{1}{4}$, the same as a random guess. All four outcomes are now equally likely. Geometrically, with $|a_\perp\rangle = \frac{1}{\sqrt{3}}(|00\rangle + |10\rangle + |11\rangle)$,

$$(WV)^2 |\Phi\rangle = -\tfrac{\sqrt{3}}{2} |a_\perp\rangle + \tfrac{1}{2} |a\rangle = \cos 150^\circ |a_\perp\rangle + \sin 150^\circ |a\rangle ,$$

i.e. the vector is at $5\theta = 150^\circ$, 60° *past* $|a\rangle$ (**Problem 5**). Every WV rotates by a fixed $2\theta = 60^\circ$, so with $N = 4$ it must be applied exactly once. More iterations do not help. This is also why the lecture's estimate $\frac{\pi}{4}\sqrt{4} \approx 1.6$ has to be rounded *down* here. The notebook (**Appendix 1**) reproduces this state, and **Problem 15** shows the uniform histogram after two iterations.

Problem 12. Numerical Substitution of Grover's Algorithm

Repeat what we did in the numerical substitution with $a = 3$.

Solution

Vectors. From **Problem 7**, with $n = 2$ and $a = 3$: $|a\rangle = |11\rangle$, $|a_\perp\rangle = \frac{1}{\sqrt{3}}(|00\rangle + |01\rangle + |10\rangle)$, $|\Phi\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$, and $\cos\theta = \frac{\sqrt{3}}{2}$, so $\theta = \frac{\pi}{6}$.

Construction of the oracle. $U_f |11\rangle = (-1)^{f(3)} |11\rangle = -|11\rangle$, and every other basis vector is unchanged. By **Problem 10**,

$$U_f = |00\rangle\langle 00| + |01\rangle\langle 01| + |10\rangle\langle 10| - |11\rangle\langle 11| = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} = \mathrm{CZ} = U_{CPS,\pi}.$$

For $a = 1$ the lecture needed X gates around the CZ to move the -1 from $|11\rangle$ to $|01\rangle$. For $a = 3$ both bits are already 1, so the oracle is the CZ gate alone. In general, an X is placed on every qubit where the bit of a is 0.

Grover diffusion operator. It does not depend on a: $W = H^{\otimes 2}\left(2|0\rangle_2\langle 0|_2 - I\right)H^{\otimes 2}$, with $2|0\rangle_2\langle 0|_2 - I = \mathrm{diag}(1,-1,-1,-1) = U_{CPS,\pi}(Z \otimes Z)$. The circuit is $H^{\otimes 2}$, then CZ (the oracle), then $H^{\otimes 2}$, $Z \otimes Z$, CZ, $H^{\otimes 2}$.

Evolution of the wavefunction.

$$H^{\otimes 2}|00\rangle = \tfrac{1}{2}\begin{pmatrix}1 & 1 & 1 & 1\end{pmatrix}^\top = |\Phi\rangle ,$$

$$V|\Phi\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \frac{1}{2}\begin{pmatrix}1\\1\\1\\1\end{pmatrix} = \frac{1}{2}\begin{pmatrix}1\\1\\1\\-1\end{pmatrix},$$

$$\begin{aligned}
&H^{\otimes 2}\left(2|0\rangle_2\langle 0|_2 - I\right)H^{\otimes 2}(V|\Phi\rangle) \\
&= \frac{1}{2}\begin{pmatrix}1&1&1&1\\1&-1&1&-1\\1&1&-1&-1\\1&-1&-1&1\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&-1&0&0\\0&0&-1&0\\0&0&0&-1\end{pmatrix}\frac{1}{2}\begin{pmatrix}1&1&1&1\\1&-1&1&-1\\1&1&-1&-1\\1&-1&-1&1\end{pmatrix}\frac{1}{2}\begin{pmatrix}1\\1\\1\\-1\end{pmatrix} \\
&= \frac{1}{4}\begin{pmatrix}1&1&1&1\\1&-1&1&-1\\1&1&-1&-1\\1&-1&-1&1\end{pmatrix}\begin{pmatrix}1&0&0&0\\0&-1&0&0\\0&0&-1&0\\0&0&0&-1\end{pmatrix}\begin{pmatrix}1\\1\\1\\-1\end{pmatrix} = \frac{1}{4}\begin{pmatrix}1&1&1&1\\1&-1&1&-1\\1&1&-1&-1\\1&-1&-1&1\end{pmatrix}\begin{pmatrix}1\\-1\\-1\\1\end{pmatrix} \\
&= \frac{1}{4}\begin{pmatrix}0\\0\\0\\4\end{pmatrix} = \begin{pmatrix}0\\0\\1\end{pmatrix} = |11\rangle = |a\rangle .
\end{aligned}$$

As for $a = 1$, one WV rotates $|\Phi\rangle$ from $\theta = \frac{\pi}{6}$ by $2\theta = \frac{\pi}{3}$ to $\frac{\pi}{2}$, exactly onto $|a\rangle = |11\rangle$. A measurement gives 11 with certainty. Only the oracle changed. $|\Phi\rangle$, W, θ, and the number of iterations are the same for every target. The notebook (**Appendix 1**) repeats each step numerically.

Problem 13. Construction of CZ Gate

Using matrix multiplication, prove that the circuit in the CZ block in Fig. 1 indeed works as a CZ gate.

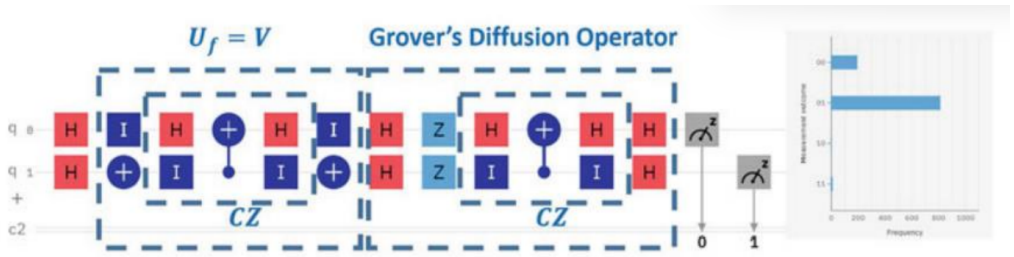


Figure 1 Circuit corresponds to the Grover's algorithm implemented in IBM-Q.
The hardware result is shown on the right.

Solution

The block. Inside each CZ box of Figure 1, q_0 (LSB, top wire) gets H, then a CNOT with control q_1 (MSB) and target q_0, then H again; q_1 only sees I. With the MSB as the first tensor factor, and the first gate on the right,

$$U_{\text{block}} = (I \otimes H)\ \text{CNOT}\ (I \otimes H), \quad I \otimes H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix}, \ \text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Matrix multiplication. CNOT on the left exchanges rows 3 and 4:

$$\text{CNOT}\,(I \otimes H) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & 1 \end{pmatrix},$$

$$(I \otimes H)\,\text{CNOT}\,(I \otimes H) = \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & 1 \end{pmatrix} = \frac{1}{2} \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & -2 \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} = \text{CZ}.$$

So the block is a CZ gate. ■ In block form the reason is clear: $\text{CNOT} = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes X$, so

$$(I \otimes H)\,\text{CNOT}\,(I \otimes H) = |0\rangle\langle 0| \otimes HIH + |1\rangle\langle 1| \otimes HXH = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes Z = \text{CZ},$$

because $HXH = Z$: the Hadamards turn a bit flip into a phase flip.

In the rest of Figure 1. In the $U_f = V$ box the CZ is sandwiched by X on q_1: $(X \otimes I)\,\text{CZ}\,(X \otimes I) = \text{diag}(1, -1, 1, 1)$, the oracle for $a = 1$ (Eq. (1) of **Problem 10**). In the diffusion box, $\text{CZ}\,(Z \otimes Z) = \text{diag}(1, -1, -1, -1) = 2\,|0\rangle_2\langle 0|_2 - I$. Figure 1's hardware histogram shows mostly `01` (about 800 of 1024), plus some `00`. The ideal output is `01` only, so the rest is hardware noise. The notebook (**Appendix 1**) checks all three products and compares the block with Qiskit's `cz`.

Problem 14. Write Your Own Grover's Algorithm Simulator

In the numerical substitution part, we have derived the matrices for all gates. Please use Google Colab to write a simulator to simulate the quantum circuit. The minimal function of your simulator should perform the matrix multiplications and give the correct output wavefunction. You may also add measurement using a random variable to see the statistics (although if you do it right, it always gives $|01\rangle$).

Solution

The simulator. The notebook (**Appendix 1**, Section 14) implements a small state-vector simulator, `MatrixSimulator`, using only NumPy matrix multiplication, so it runs unchanged in Google Colab.

- The state of n qubits is a 2^n column vector, starting at $|0\rangle_n$. Qubit 0 is the LSB, and the MSB is the first tensor factor (IBM-Q convention).
- A one-qubit gate on qubit k is expanded to the full space as $I \otimes \cdots \otimes U \otimes \cdots \otimes I$. CNOT is built from projectors, $|0\rangle\langle 0|_c \otimes I + |1\rangle\langle 1|_c \otimes X_t$.
- Every gate multiplies the state on the left, $|\psi\rangle \to U|\psi\rangle$, and the state after each step is stored.
- A measurement samples 1024 outcomes from the probabilities $|\psi_x|^2$ with a random generator (`numpy.random.Generator.choice`).

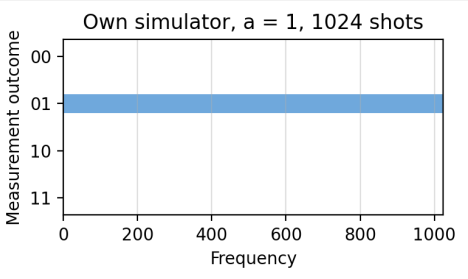
```
sim = MatrixSimulator(2)
sim.layer({0: H, 1: H}, "H (x) H")                # |Phi>
sim.layer({1: X}, "oracle: X")                    # U_f = (X(x)I) CZ (X(x)I),  CZ = (I(x)H)
    CNOT (I(x)H)
sim.layer({0: H}, ...); sim.apply(sim.cnot(1, 0), ...); sim.layer({0: H}, ...)
sim.layer({1: X}, "oracle: X")
sim.layer({0: H, 1: H}, ...); sim.layer({0: Z, 1: Z}, ...)                    # W
sim.layer({0: H}, ...); sim.apply(sim.cnot(1, 0), ...); sim.layer({0: H}, ...)
sim.layer({0: H, 1: H}, ...)
counts = sim.measure(1024, rng)
```

Output wavefunction. Running Figure 1 gate by gate ($a = 1$) gives the following states, in the basis $|00\rangle, |01\rangle, |10\rangle, |11\rangle$:

after	state
$H \otimes H$	$\frac{1}{2}(1,1,1,1) = \|\Phi\rangle$
oracle $U_f = V$	$\frac{1}{2}(1,-1,1,1)$
$H \otimes H$	$\frac{1}{2}(1,1,-1,1)$
$\mathrm{CZ}\,(Z \otimes Z)$	$\frac{1}{2}(1,-1,1,-1)$
$H \otimes H$	$(0,1,0,0) = \|01\rangle$

These match the numerical substitution of the lecture step by step. Inside each CZ block the intermediate states are, for example, $\frac{1}{\sqrt{2}}(1,0,1,0) \to \frac{1}{\sqrt{2}}(1,0,0,1)$ in the oracle (full trace in the appendix).

Measurement. Sampling 1024 shots gives `01` every time (figure below). Running the simulator with the oracle for each of $a = 0, 1, 2, 3$ gives `00`, `01`, `10`, `11` in all 1024 shots, respectively. Each final state vector was also compared with Qiskit's `Statevector` of the same circuit, and they agree.



Problem 15. Implement Grover's Algorithm with XOR Gate

Implement the circuit in Fig. 2 on IBM-Q. You need to first construct the corresponding XOR oracle. Set $a = 1$. Note that this is a 3-qubit circuit due to the addition of the auxiliary bit.

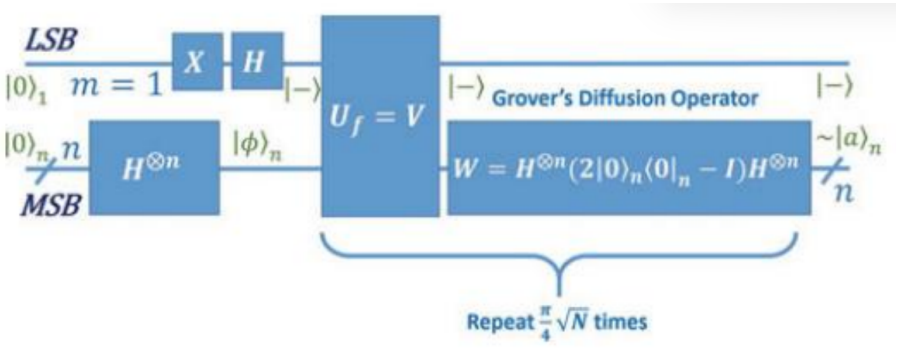


Figure 2 Circuit to implement the Grover's algorithm with a XOR quantum oracle, U_f.

Solution

Setting. $n = 2$ query qubits hold x (q_2 = MSB, q_1), and $m = 1$ auxiliary qubit y is the LSB q_0, as in Figure 2. The basis vectors are $|x\rangle_2 |y\rangle_1 = |q_2 q_1 q_0\rangle$, and $a = 1$, so $f(01) = 1$ and $f(x) = 0$ otherwise.

Construction of the XOR oracle. From $U_f |x\rangle_2 |y\rangle = |x\rangle_2 |y \oplus f(x)\rangle$, only $x = 01$ changes:

$$U_f |01\rangle |0\rangle = |01\rangle |0 \oplus 1\rangle = |01\rangle |1\rangle , \qquad U_f |01\rangle |1\rangle = |01\rangle |1 \oplus 1\rangle = |01\rangle |0\rangle ,$$

and $U_f |x\rangle |y\rangle = |x\rangle |y\rangle$ for $x \in \{00, 10, 11\}$. With $U = \sum_{i,j} \langle j| U |i\rangle |j\rangle \langle i|$, the non-zero elements are $\langle 011| U_f |010\rangle = \langle 010| U_f |011\rangle = 1$ and the six diagonal ones for the other basis vectors:

$$U_f = \sum_{i \notin \{010,011\}} |i\rangle \langle i| + |011\rangle \langle 010| + |010\rangle \langle 011| = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} .$$

It flips q_0 only when $q_2 = 0$ and $q_1 = 1$. That is a Toffoli gate (controls q_2, q_1, target q_0) whose q_2 control is made to fire on 0 by X gates before and after: $U_f = (X \otimes I \otimes I)$ CCX $(X \otimes I \otimes I)$. With the auxiliary qubit in $|-\rangle$, phase kickback turns it into the phase oracle, $U_f |x\rangle_2 |-\rangle = (-1)^{f(x)} |x\rangle_2 |-\rangle = (V \otimes I) |x\rangle_2 |-\rangle$, with $V = \mathrm{diag}(1, -1, 1, 1)$ (lecture, Chapter 23).

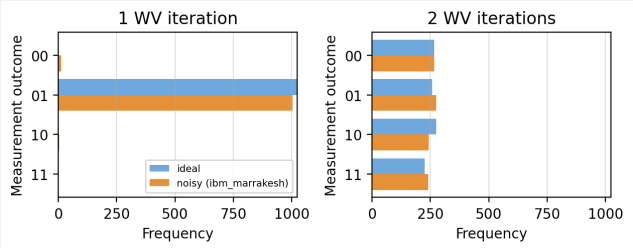
The circuit. X then H on q_0 prepares $|-\rangle$, and $H^{\otimes 2}$ on q_2, q_1 prepares $|\Phi\rangle_2$. Then come U_f and W on the query qubits, built as in **Problems 12** and **13**. The number of repetitions: $\frac{\pi}{4}\sqrt{N} = \frac{\pi}{2} \approx 1.6$, but $\theta = 30°$ exactly, so one WV reaches $|a\rangle$ and a second one over-rotates (**Problem 11**). We use one.

```
qc.x(0); qc.h(0)                                          # auxiliary |0> -> |->
qc.h([1, 2])                                              # |Phi>_2
qc.x(2); qc.ccx(2, 1, 0); qc.x(2)                         # XOR oracle U_f for a = 1
qc.h([1, 2]); qc.z([1, 2]); qc.cz(2, 1); qc.h([1, 2])     # W
qc.measure([1, 2], [0, 1])                                # bit string q2 q1 = x
```

Before the measurement the state is exactly $|01\rangle_2 |-\rangle$ (checked with Qiskit's `Statevector`). The auxiliary qubit comes out unchanged, as the lecture predicts.

Results. As in **Problem 4**, the circuit was run on Qiskit's `AerSimulator` instead of IBM-Q hardware: ideal, and with the `ibm_marrakesh` noise model (1024 shots each). After transpiling for `ibm_marrakesh`, the circuit has depth 50 and 10 two-qubit (CZ) gates, most of them from the Toffoli.

counts q_2q_1 (1024 shots)	`00`	`01`	`10`	`11`
1 iteration, ideal	0	1024	0	0
1 iteration, `ibm_marrakesh` noise	13	1003	5	3
2 iterations, ideal	267	257	275	225
2 iterations, `ibm_marrakesh` noise	267	274	243	240



With one iteration, Grover's algorithm with the XOR oracle finds $a = 01$ in every ideal shot and in 98% of the noisy shots. This is the same answer as the phase-oracle circuit of Figure 1. With two iterations the state is over-rotated to 150°, and all four outcomes appear with probability $\frac{1}{4}$ each, as computed in **Problem 11**.

> **Remark.** The auxiliary qubit can also be measured. Adding H and then X on q_0 at the end returns $|-\rangle \to |1\rangle \to |0\rangle$, so the full 3-qubit outcome should always be $|q_2q_1q_0\rangle = |010\rangle$. The simulator gives `010` in all 1024 ideal shots and in 987 noisy shots. The auxiliary qubit really is left in $|-\rangle$, not entangled with the query register.

APPENDIX 1

Assignment 09 | Code Solution

assignment-09-code

October 24, 2026

1 Assignment 09

1.1 Import External Libraries

[1]:
```
import numpy as np
import sympy as sp
import matplotlib.pyplot as plt

from IPython.display import display, Math

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister, transpile
from qiskit.circuit.library import SwapGate
from qiskit.quantum_info import Operator, Statevector
from qiskit_aer import AerSimulator
from qiskit_ibm_runtime.fake_provider import FakeMarrakesh
```

[2]:
```
# --- Shared helpers: qubit order |q_{n-1} ... q_1 q_0>, MSB = first tensor factor (IBM-Q convention) ---
def latex_matrix(M):
    """LaTeX for a numeric array: rounded to 4 d.p., whole numbers shown as integers."""
    M = np.round(np.real_if_close(np.asarray(M, dtype=complex)), 4) + 0.0
    return sp.latex(sp.Matrix(M).applyfunc(lambda v: sp.Integer(int(v)) if float(v).is_integer() else v))


def ket(bits):
    """Basis column vector |bits>, e.g. ket("01") = (0, 1, 0, 0)."""
    v = np.zeros(2 ** len(bits))
    v[int(bits, 2)] = 1
    return v


def kron(*ms):
    out = np.array([[1.0]])
    for m in ms:
        out = np.kron(out, m)
    return out


I2 = np.eye(2)
X  = np.array([[0, 1], [1, 0]])
Z  = np.array([[1, 0], [0, -1]])
H  = np.array([[1, 1], [1, -1]]) / np.sqrt(2)
P0 = np.array([[1, 0], [0, 0]])   # |0><0|
P1 = np.array([[0, 0], [0, 1]])   # |1><1|

CNOT = kron(P0, I2) + kron(P1, X)   # control = MSB q1, target = LSB q0
CZ   = kron(P0, I2) + kron(P1, Z)
HH   = kron(H, H)
```

```
# Simulators: ideal, and with the noise model of IBM's ibm_marrakesh (a snapshot of its calibration)
SHOTS   = 1024
SEED    = 2110515
ideal   = AerSimulator(seed_simulator=SEED)
backend = FakeMarrakesh()
noisy   = AerSimulator.from_backend(backend, seed_simulator=SEED)


def run(qc, sim):
    tqc = transpile(qc, sim, optimization_level=1, seed_transpiler=SEED)
    return sim.run(tqc, shots=SHOTS).result().get_counts()


def report(cnt, label, nbits):
    print(label)
    for k in range(2 ** nbits):
        key = format(k, f"0{nbits}b")
        print(f"  {key}:  {cnt.get(key, 0):4d} / {SHOTS}")


def histograms(panels, filename, nbits, ncols=None):
    """One horizontal histogram per panel; each panel = (title, ideal counts, noisy counts)."""
    ncols = ncols or len(panels)
    nrows = -(-len(panels) // ncols)
    fig, axes = plt.subplots(nrows, ncols, figsize=(3.2 * ncols, 2.6 * nrows), squeeze=False)
    keys = [format(k, f"0{nbits}b") for k in range(2 ** nbits)][::-1]
    y = np.arange(len(keys))
    for ax, (title, ci, cn) in zip(axes.flat, panels):
        ax.barh(y + 0.2, [ci.get(k, 0) for k in keys], height=0.4, color="#6fa8dc", label="ideal")
        ax.barh(y - 0.2, [cn.get(k, 0) for k in keys], height=0.4, color="#e69138", label="noisy (ibm_marrakesh)")
        ax.set_yticks(y, keys)
        ax.set_xlim(0, SHOTS)
        ax.set_xlabel("Frequency")
        ax.set_ylabel("Measurement outcome")
        ax.set_title(title)
        ax.grid(axis="x", alpha=0.4)
    for ax in list(axes.flat)[len(panels):]:
        ax.axis("off")
    axes.flat[0].legend(loc="lower right", fontsize=7)
    fig.tight_layout()
    fig.savefig(f"../images/{filename}", dpi=200, bbox_inches="tight")
    display(fig)
    plt.close(fig)
```

1.2 1. The General Form of XOR Quantum Oracle

[3]:
```
# --- Problem 1: x = 5, f(5) = 3, n = 3, m = 3 ---
n, m, x, fx, y = 3, 3, 5, 3, 0
x_bits, y_bits  = format(x, f"0{n}b"), format(y, f"0{m}b")
out_bits        = format(y ^ fx, f"0{m}b")
print(f"Problem 1  --  input  |x>_3|y>_3 = |{x_bits}>|{y_bits}> = |{x_bits + y_bits}>  (decimal {int(x_bits + y_bits, 2)})")
print(f"Problem 1  --  output |x>_3|y XOR f(x)>_3 = |{x_bits}>|{out_bits}> = |{x_bits + out_bits}>  (decimal {int(x_bits + out_bits, 2)})")
```

```
Problem 1  --  input  |x>_3|y>_3 = |101>|000> = |101000>  (decimal 40)
```

```
Problem 1  --  output |x>_3|y XOR f(x)>_3 = |101>|011> = |101011>  (decimal 43)
```

1.3 2. Unitarity of XOR Quantum Oracle

[4]:
```
# --- Problem 2: U_f is unitary for ANY f: {0,1}^n -> {0,1}^m, even a non-injective one ---
def xor_oracle(f, n, m):
    """Matrix of U_f |x>_n |y>_m = |x>_n |y XOR f(x)>_m  (x = the n MSB, y = the m LSB)."""
    N = 2 ** (n + m)
    U = np.zeros((N, N))
    for xx in range(2 ** n):
        for yy in range(2 ** m):
            U[(xx << m) | (yy ^ f(xx)), (xx << m) | yy] = 1   # column |x>|y>  ->  row |x>|y XOR␣
 ↪f(x)>
    return U


rng = np.random.default_rng(SEED)
for trial in range(5):
    table = [int(v) for v in rng.integers(0, 2 ** 3, size=2 ** 3)]   # a random f: {0,1}^3 ->␣
 ↪{0,1}^3
    U = xor_oracle(lambda xx: table[xx], 3, 3)
    print(f"Problem 2  --  f = {table}:  U^dagger U == I ? {np.allclose(U.T @ U, np.eye(64))},"
          f"  U U == I ? {np.allclose(U @ U, np.eye(64))}")
```

```
Problem 2  --  f = [2, 4, 4, 0, 2, 3, 4, 6]:  U^dagger U == I ? True,  U U == I
? True
Problem 2  --  f = [3, 7, 6, 1, 2, 1, 7, 0]:  U^dagger U == I ? True,  U U == I
? True
Problem 2  --  f = [0, 5, 6, 4, 3, 6, 3, 1]:  U^dagger U == I ? True,  U U == I
? True
Problem 2  --  f = [2, 7, 2, 0, 0, 5, 1, 4]:  U^dagger U == I ? True,  U U == I
? True
Problem 2  --  f = [5, 6, 1, 7, 2, 4, 6, 3]:  U^dagger U == I ? True,  U U == I
? True
```

1.4 3. Construction of SWAP-gate

[5]:
```
# --- Problem 3: SWAP = sum_{i,j} <j|SWAP|i> |j><i| ---
basis = ["00", "01", "10", "11"]
swap_action = {"00": "00", "01": "10", "10": "01", "11": "11"}   # SWAP|q1 q0> = |q0 q1>

SWAP = np.zeros((4, 4))
for i in basis:
    for j in basis:
        coeff = ket(j) @ ket(swap_action[i])                    # <j| SWAP |i>
        if coeff:
            print(f"Problem 3  --  <{j}|SWAP|{i}> = {coeff:.0f}")
        SWAP += coeff * np.outer(ket(j), ket(i))

display(Math(r"\text{SWAP} = " + latex_matrix(SWAP)))
print("Problem 3  --  equals Qiskit's SwapGate ?", np.allclose(SWAP, Operator(SwapGate()).data))
```

```
Problem 3  --  <00|SWAP|00> = 1
Problem 3  --  <10|SWAP|01> = 1
Problem 3  --  <01|SWAP|10> = 1
Problem 3  --  <11|SWAP|11> = 1
```

$$\text{SWAP} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 3  --  equals Qiskit's SwapGate ? True
```

1.5 4. Deutsch Algorithm

[6]:
```python
# --- Problem 4: XOR oracles of f_A ... f_D (x = MSB q1, y = LSB q0) and their gate circuits ---
functions = {"A": (0, 0), "B": (0, 1), "C": (1, 0), "D": (1, 1)}   # (f(0), f(1))


def oracle_circuit(name):
    qc = QuantumCircuit(2, name=f"U_f{name}")
    if name == "B":
        qc.cx(1, 0)                 # y XOR x
    elif name == "C":
        qc.cx(1, 0)                 # y XOR x ...
        qc.x(0)                     # ... XOR 1  =  y XOR f_C(x), since f_C(x) = 1 XOR x
    elif name == "D":
        qc.x(0)                     # y XOR 1
    return qc                       # f_A: identity


for name, (f0, f1) in functions.items():
    U = xor_oracle(lambda xx: (f0, f1)[xx], 1, 1)
    display(Math(rf"U_{{f_{name}}} = " + latex_matrix(U)))
    print(f"Problem 4  --  f_{name}: gate circuit matches the matrix ?",
          np.allclose(Operator(oracle_circuit(name)).data, U))
```

$$U_{f_A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 4  --  f_A: gate circuit matches the matrix ? True
```

$$U_{f_B} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

```
Problem 4  --  f_B: gate circuit matches the matrix ? True
```

$$U_{f_C} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 4  --  f_C: gate circuit matches the matrix ? True
```

$$U_{f_D} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

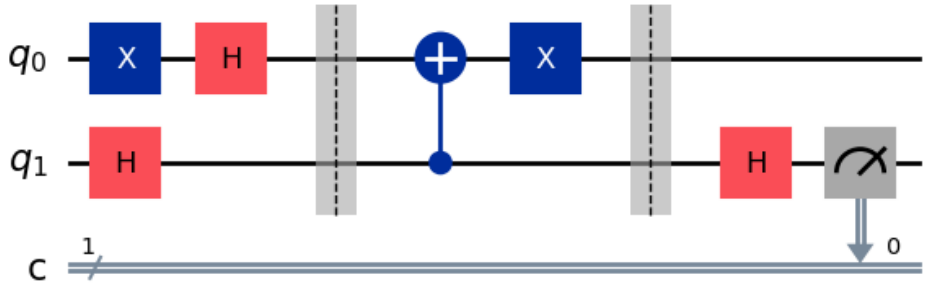
```
Problem 4  --  f_D: gate circuit matches the matrix ? True
```

[7]:
```python
# --- Problem 4: the Deutsch circuit with each oracle, ideal and with ibm_marrakesh noise ---
def deutsch(name):
    qc = QuantumCircuit(QuantumRegister(2, "q"), ClassicalRegister(1, "c"))
    qc.x(0)                         # auxiliary qubit |1> ...
    qc.h([0, 1])                    # ... -> |->, and x -> |+>
```

```
    qc.barrier()
    qc.compose(oracle_circuit(name), inplace=True)
    qc.barrier()
    qc.h(1)
    qc.measure(1, 0)                # 0 = constant, 1 = balanced
    return qc


display(deutsch("C").draw("mpl"))
plt.close("all")

panels = []
for name, (f0, f1) in functions.items():
    ci, cn = run(deutsch(name), ideal), run(deutsch(name), noisy)
    kind = "constant" if f0 == f1 else "balanced"
    report(ci, f"f_{name} ({kind}), ideal", 1)
    report(cn, f"f_{name} ({kind}), noisy", 1)
    panels.append((rf"$f_{name}$ ({kind})", ci, cn))
histograms(panels, "problem-4-answer.png", 1, ncols=2)
```

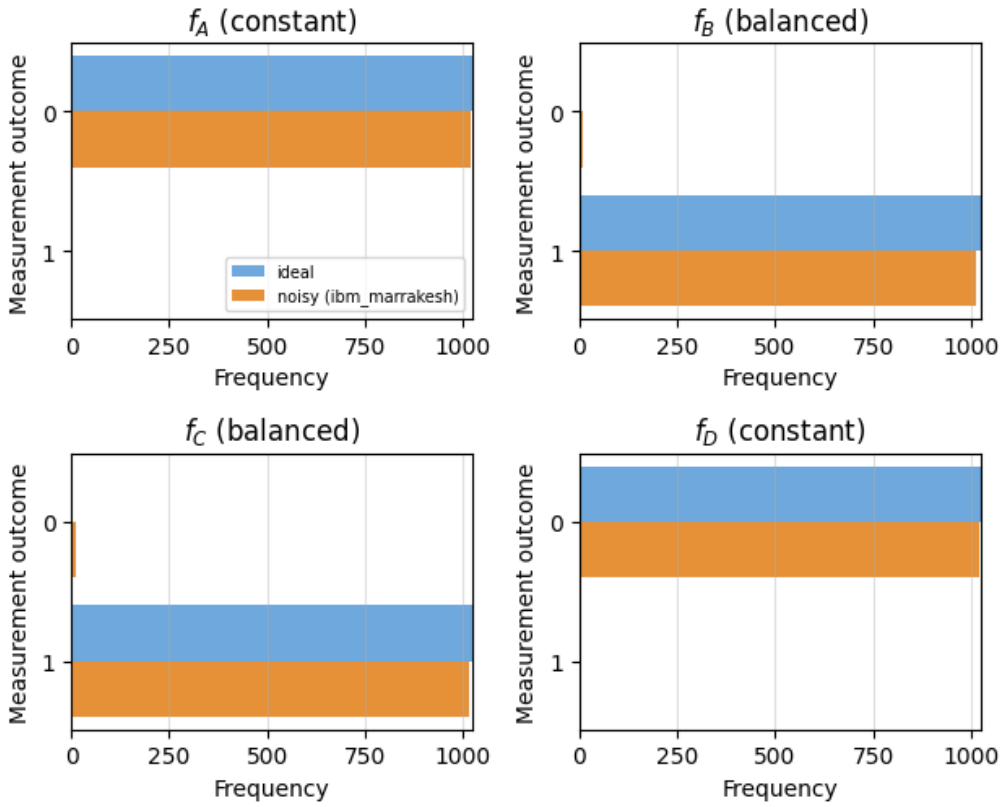



```
f_A (constant), ideal
  0:  1024 / 1024
  1:     0 / 1024
f_A (constant), noisy
  0:  1020 / 1024
  1:     4 / 1024

f_B (balanced), ideal
  0:     0 / 1024
  1:  1024 / 1024
f_B (balanced), noisy
  0:    10 / 1024
  1:  1014 / 1024

f_C (balanced), ideal
  0:     0 / 1024
  1:  1024 / 1024
f_C (balanced), noisy
  0:    10 / 1024
  1:  1014 / 1024

f_D (constant), ideal
  0:  1024 / 1024
  1:     0 / 1024
f_D (constant), noisy
  0:  1020 / 1024
  1:     4 / 1024
```



1.6 5. Rotation of $|\Phi$

[8]:
```
# --- Problem 5: signed angles of the rotated |Phi> (from |a_perp> towards |a>), N = 4, a = 1 ---
n, a = 2, 1
N = 2 ** n
phi    = np.ones(N) / np.sqrt(N)
a_vec  = ket(format(a, f"0{n}b"))
a_perp = (np.ones(N) - a_vec) / np.sqrt(N - 1)
V = np.eye(N) - 2 * np.outer(a_vec, a_vec)
W = 2 * np.outer(phi, phi) - np.eye(N)
theta = np.degrees(np.arcsin(1 / np.sqrt(N)))


def angle_from_a_perp(v):
    return np.degrees(np.arctan2(a_vec @ v, a_perp @ v))


# (WV)^k |Phi> sits at (2k+1) theta, V (WV)^k |Phi> at -(2k+1) theta; check each against the␣
  ↪state vector
print(f"theta = {theta:.1f} deg")
print(f"{'state':>14} {'to |a_perp>':>12} {'to |a>':>8}   matches the state vector?")
state, label = phi, "|Phi>"
for step in range(9):
    k = step // 2
    formula = (2 * k + 1) * theta * (1 if step % 2 == 0 else -1)
    same = np.isclose((formula - angle_from_a_perp(state) + 180) % 360, 180)
    print(f"{label:>14} {formula:12.1f} {round(90 - formula, 6) + 0.0:8.1f}   {same}")
```

```
    op, name = (V, "V") if step % 2 == 0 else (W, "W")
    state, label = op @ state, name + label
```

```
theta = 30.0 deg
         state  to |a_perp>   to |a>   matches the state vector?
         |Phi>         30.0     60.0   True
        V|Phi>        -30.0    120.0   True
       WV|Phi>         90.0      0.0   True
      VWV|Phi>        -90.0    180.0   True
     WVWV|Phi>        150.0    -60.0   True
    VWVWV|Phi>       -150.0    240.0   True
   WVWVWV|Phi>        210.0   -120.0   True
  VWVWVWV|Phi>       -210.0    300.0   True
 WVWVWVWV|Phi>        270.0   -180.0   True
```

1.7 6. Another Form of V

[9]:
```
# --- Problem 6: V = I - 2|a><a|  and  W = 2|Phi><Phi| - I = H (2|0><0| - I) H  (n = 2) ---
n = 2
N = 2 ** n
phi = np.ones(N) / np.sqrt(N)
for a in range(N):
    a_vec = ket(format(a, f"0{n}b"))
    V_def = np.diag([(-1) ** (x == a) for x in range(N)])           # V|x> = (-1)^{f(x)} |x>
    print(f"Problem 6  --  a = {a}:  V == I - 2|a><a| ?", np.allclose(V_def, np.eye(N) - 2 * np.
  ↪outer(a_vec, a_vec)))

W = 2 * np.outer(phi, phi) - np.eye(N)
print("Problem 6  --  W == H (2|00><00| - I) H ?", np.allclose(W, HH @ (2 * np.outer(ket("00"),␣
  ↪ket("00")) - np.eye(N)) @ HH))
print("Problem 6  --  W == -(I - 2|Phi><Phi|) ?", np.allclose(W, -(np.eye(N) - 2 * np.outer(phi,␣
  ↪phi))))
```

```
Problem 6  --  a = 0:  V == I - 2|a><a| ? True
Problem 6  --  a = 1:  V == I - 2|a><a| ? True
Problem 6  --  a = 2:  V == I - 2|a><a| ? True
Problem 6  --  a = 3:  V == I - 2|a><a| ? True
Problem 6  --  W == H (2|00><00| - I) H ? True
Problem 6  --  W == -(I - 2|Phi><Phi|) ? True
```

1.8 7. Numerical Substitution I

[10]:
```
# --- Problem 7: n = 2, a = 3 ---
n, a = 2, 3
N = 2 ** n
phi    = np.ones(N) / np.sqrt(N)
a_vec  = ket(format(a, f"0{n}b"))
a_perp = (np.ones(N) - a_vec) / np.sqrt(N - 1)
display(Math(r"\ket{\Phi} = " + sp.latex(sp.Matrix([sp.Rational(1, 2)] * 4).T)))
display(Math(r"\ket{a} = " + latex_matrix(a_vec.reshape(1, -1))))
display(Math(r"\ket{a_\perp} = \tfrac{1}{\sqrt{3}}" + latex_matrix((np.sqrt(3) * a_perp).
  ↪reshape(1, -1))))
print("Problem 7  --  <a|a_perp> =", round(a_vec @ a_perp, 10), "  cos(theta) = <a_perp|Phi> =",␣
  ↪round(a_perp @ phi, 4),
      "  theta =", round(np.degrees(np.arccos(a_perp @ phi)), 2), "deg")
```

$|\Phi\rangle = \begin{bmatrix}\frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2}\end{bmatrix}$

$|a\rangle = \begin{bmatrix}0 & 0 & 0 & 1\end{bmatrix}$

$|a_\perp\rangle = \frac{1}{\sqrt{3}}\begin{bmatrix}1 & 1 & 1 & 0\end{bmatrix}$

```
Problem 7  --  <a|a_perp> = 0.0   cos(theta) = <a_perp|Phi> = 0.866   theta =
30.0 deg
```

1.9 8. Numerical Substitution II

[11]:
```
# --- Problem 8: V|Phi> = c0 |Phi> + c1 |Phi_perp>  (continuing Problem 7, a = 3) ---
theta    = np.arccos(a_perp @ phi)
phi_perp = -np.sin(theta) * a_perp + np.cos(theta) * a_vec
V        = np.eye(N) - 2 * np.outer(a_vec, a_vec)
V_phi    = V @ phi
c0, c1   = phi @ V_phi, phi_perp @ V_phi
display(Math(r"\ket{\Phi_\perp} = \tfrac{1}{2\sqrt{3}}" + latex_matrix((2 * np.sqrt(3) *␣
  ↪phi_perp).reshape(1, -1))))
display(Math(r"V\ket{\Phi} = " + latex_matrix(V_phi.reshape(1, -1))))
print(f"Problem 8  --  c0 = <Phi|V|Phi> = {c0:.4f},  c1 = <Phi_perp|V|Phi> = {c1:.4f}  (sqrt3/2 =␣
  ↪{np.sqrt(3) / 2:.4f})")
print("Problem 8  --  V|Phi> == c0|Phi> + c1|Phi_perp> ?", np.allclose(V_phi, c0 * phi + c1 *␣
  ↪phi_perp))
```

$|\Phi_\perp\rangle = \frac{1}{2\sqrt{3}}\begin{bmatrix}-1 & -1 & -1 & 3\end{bmatrix}$

$V|\Phi\rangle = \begin{bmatrix}0.5 & 0.5 & 0.5 & -0.5\end{bmatrix}$

```
Problem 8  --  c0 = <Phi|V|Phi> = 0.5000,  c1 = <Phi_perp|V|Phi> = -0.8660
(sqrt3/2 = 0.8660)
Problem 8  --  V|Phi> == c0|Phi> + c1|Phi_perp> ? True
```

1.10 9. Computational Complexity

[12]:
```
# --- Problem 9: O(e^N) vs O(2N): the ratio grows exponentially ---
print(f"{'N':>4} {'e^N':>14} {'2N':>5} {'e^N / 2N':>14}")
for Nsize in (1, 5, 10, 20, 40):
    print(f"{Nsize:>4} {np.exp(Nsize):14.4g} {2 * Nsize:>5} {np.exp(Nsize) / (2 * Nsize):14.4g}")
```

```
   N            e^N    2N       e^N / 2N
   1          2.718     2          1.359
   5          148.4    10          14.84
  10      2.203e+04    20           1101
  20      4.852e+08    40       1.213e+07
  40      2.354e+17    80       2.942e+15
```

1.11 10. Construction of Oracle

[13]:
```
# --- Problem 10: U_f = sum_{i,j} <j|V|i> |j><i|  with  V|x> = (-1)^{f(x)} |x>, a = 1 ---
f = lambda x: int(x == 1)
basis = ["00", "01", "10", "11"]
U_f = np.zeros((4, 4))
for i in basis:
    V_i = (-1) ** f(int(i, 2)) * ket(i)                    # V|i>
    for j in basis:
        coeff = ket(j) @ V_i                               # <j|V|i>
        if coeff:
            print(f"Problem 10  --  <{j}|V|{i}> = {coeff:+.0f}")
        U_f += coeff * np.outer(ket(j), ket(i))
display(Math(r"U_f = " + latex_matrix(U_f)))
```

```
print("Problem 10  --  U_f == (X (x) I) CZ (X (x) I) ?", np.allclose(U_f, kron(X, I2) @ CZ @ kron(X, I2)))
```

```
Problem 10  --  <00|V|00> = +1
Problem 10  --  <01|V|01> = -1
Problem 10  --  <10|V|10> = +1
Problem 10  --  <11|V|11> = +1
```

$$U_f = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 10  --  U_f == (X (x) I) CZ (X (x) I) ? True
```

1.12 11. Rotation of VW|Φ

[14]:
```
# --- Problem 11: one more WV on |a> = |01> ---
V = np.diag([1, -1, 1, 1])
W = HH @ np.diag([1, -1, -1, -1]) @ HH
state = W @ V @ (HH @ ket("00"))                # one WV: |01>
display(Math(r"WV\ket{\Phi} = " + latex_matrix(state.reshape(1, -1))))
state2 = W @ V @ state
display(Math(r"V\,WV\ket{\Phi} = " + latex_matrix((V @ state).reshape(1, -1))))
display(Math(r"WV\,WV\ket{\Phi} = " + latex_matrix(state2.reshape(1, -1))))
print("Problem 11  --  probabilities |00>,|01>,|10>,|11>:", np.round(np.abs(state2) ** 2, 4))
a_perp = (ket("00") + ket("10") + ket("11")) / np.sqrt(3)
print("Problem 11  --  angle from |a_perp> =", round(float(np.degrees(np.arctan2(ket("01") @ state2, a_perp @ state2))), 2), "deg")
```

$$WV\,|\Phi\rangle = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$$

$$V\,WV\,|\Phi\rangle = \begin{bmatrix} 0 & -1 & 0 & 0 \end{bmatrix}$$

$$WV\,WV\,|\Phi\rangle = \begin{bmatrix} -0.5 & 0.5 & -0.5 & -0.5 \end{bmatrix}$$

```
Problem 11  --  probabilities |00>,|01>,|10>,|11>: [0.25 0.25 0.25 0.25]
Problem 11  --  angle from |a_perp> = 150.0 deg
```

1.13 12. Numerical Substitution of Grover's Algorithm

[15]:
```
# --- Problem 12: the whole numerical example with a = 3 ---
U_f = np.diag([1, 1, 1, -1])
print("Problem 12  --  U_f == CZ ?", np.allclose(U_f, CZ))
print("Problem 12  --  2|00><00| - I == CZ (Z (x) Z) ?", np.allclose(np.diag([1, -1, -1, -1]), CZ @ kron(Z, Z)))

steps = [("H^{\\otimes 2}\\ket{00}", HH), ("V\\ket{\\Phi}", U_f), ("H^{\\otimes 2}V\\ket{\\Phi}", HH),
         ("(2\\ket{0}\\bra{0}-I)\\cdots", np.diag([1, -1, -1, -1])), ("WV\\ket{\\Phi}", HH)]
state = ket("00")
for label, M in steps:
    state = M @ state
    display(Math(label + " = " + latex_matrix(state.reshape(1, -1))))
print("Problem 12  --  final state == |11> ?", np.allclose(state, ket("11")))
```

```
Problem 12  --  U_f == CZ ? True
Problem 12  --  2|00><00| - I == CZ (Z (x) Z) ? True
```

$$H^{\otimes 2}\,|00\rangle = \begin{bmatrix} 0.5 & 0.5 & 0.5 & 0.5 \end{bmatrix}$$

$V|\Phi\rangle = [0.5 \quad 0.5 \quad 0.5 \quad -0.5]$

$H^{\otimes 2}V|\Phi\rangle = [0.5 \quad 0.5 \quad 0.5 \quad -0.5]$

$(2|0\rangle\langle 0| - I)\cdots = [0.5 \quad -0.5 \quad -0.5 \quad 0.5]$

$WV|\Phi\rangle = [0 \quad 0 \quad 0 \quad 1]$

```
Problem 12  --  final state == |11> ? True
```

1.14 13. Construction of CZ Gate

[16]:
```
# --- Problem 13: the CZ block of Figure 1 = (I (x) H) CNOT (I (x) H)  (H on the target q0 = LSB)
# ---
IH = kron(I2, H)
display(Math(r"I \otimes H = \tfrac{1}{\sqrt{2}}" + latex_matrix(np.sqrt(2) * IH)))
display(Math(r"\text{CNOT}\,(I \otimes H) = \tfrac{1}{\sqrt{2}}" + latex_matrix(np.sqrt(2) * CNOT
@ IH)))
block = IH @ CNOT @ IH
display(Math(r"(I \otimes H)\,\text{CNOT}\,(I \otimes H) = " + latex_matrix(block)))
print("Problem 13  --  block == CZ ?", np.allclose(block, CZ))
qc_cz = QuantumCircuit(2)
qc_cz.cz(1, 0)
print("Problem 13  --  equals Qiskit's cz ?", np.allclose(block, Operator(qc_cz).data))
print("Problem 13  --  Figure 1's U_f = (X (x) I) CZ (X (x) I) == diag(1, -1, 1, 1) ?",
      np.allclose(kron(X, I2) @ block @ kron(X, I2), np.diag([1, -1, 1, 1])))
```

$$I \otimes H = \tfrac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & -1 \end{bmatrix}$$

$$\mathrm{CNOT}\,(I \otimes H) = \tfrac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

$$(I \otimes H)\,\mathrm{CNOT}\,(I \otimes H) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$$

```
Problem 13  --  block == CZ ? True
Problem 13  --  equals Qiskit's cz ? True
Problem 13  --  Figure 1's U_f = (X (x) I) CZ (X (x) I) == diag(1, -1, 1, 1) ?
True
```

1.15 14. Write Your Own Grover's Algorithm Simulator

[17]:
```
# --- Problem 14: a matrix-multiplication simulator (numpy only; runs unchanged in Google Colab)
# ---
class MatrixSimulator:
    """n-qubit state-vector simulator. Qubit 0 is the LSB; the MSB is the first tensor factor."""

    def __init__(self, n):
        self.n = n
        self.state = np.zeros(2 ** n)
        self.state[0] = 1                          # |00...0>
        self.history = [("start", self.state.copy())]

    def single(self, gate, qubit):
```

```
        """Matrix of a one-qubit gate on `qubit`, identity elsewhere."""
        return kron(*[gate if q == qubit else I2 for q in reversed(range(self.n))])

    def cnot(self, control, target):
        """CNOT = |0><0|_c (x) I + |1><1|_c (x) X_t, built from projectors."""
        ops0 = [P0 if q == control else I2 for q in reversed(range(self.n))]
        ops1 = [P1 if q == control else (X if q == target else I2) for q in reversed(range(self.
 ↪n))]
        return kron(*ops0) + kron(*ops1)

    def apply(self, U, label):
        self.state = U @ self.state
        self.history.append((label, self.state.copy()))
        return self

    def layer(self, gates, label):
        """One time step: a dict {qubit: 2x2 gate}; missing qubits get I."""
        return self.apply(kron(*[gates.get(q, I2) for q in reversed(range(self.n))]), label)

    def measure(self, shots, rng):
        """Sample `shots` outcomes with probabilities |amplitude|^2 (Born rule)."""
        p = np.abs(self.state) ** 2
        outcomes = rng.choice(2 ** self.n, size=shots, p=p / p.sum())
        keys, cnt = np.unique(outcomes, return_counts=True)
        return {format(k, f"0{self.n}b"): int(c) for k, c in zip(keys, cnt)}


def figure1(a):
    """Figure 1 (Pic. 31) gate by gate, for the target |a> = |a1 a0> (q1 = MSB, q0 = LSB)."""
    sim = MatrixSimulator(2)
    flips = {q: X for q in (0, 1) if not (a >> q) & 1}     # X where the bit of a is 0 (I for a␣
 ↪bit 1)
    sim.layer({0: H, 1: H}, "H (x) H")
    # U_f = V: X's, CZ = (I (x) H) CNOT (I (x) H), X's
    sim.layer(flips, "oracle: X's")
    sim.layer({0: H}, "oracle: CZ - H on q0")
    sim.apply(sim.cnot(1, 0), "oracle: CZ - CNOT")
    sim.layer({0: H}, "oracle: CZ - H on q0")
    sim.layer(flips, "oracle: X's")
    # W = H (2|0><0| - I) H with 2|0><0| - I = CZ (Z (x) Z)
    sim.layer({0: H, 1: H}, "W: H (x) H")
    sim.layer({0: Z, 1: Z}, "W: Z (x) Z")
    sim.layer({0: H}, "W: CZ - H on q0")
    sim.apply(sim.cnot(1, 0), "W: CZ - CNOT")
    sim.layer({0: H}, "W: CZ - H on q0")
    sim.layer({0: H, 1: H}, "W: H (x) H")
    return sim


sim = figure1(a=1)
for label, s in sim.history:
    print(f"{label:>22}:  {np.round(s, 4)}")
print("Problem 14  --  final state == |01> ?", np.allclose(sim.state, ket("01")))
```

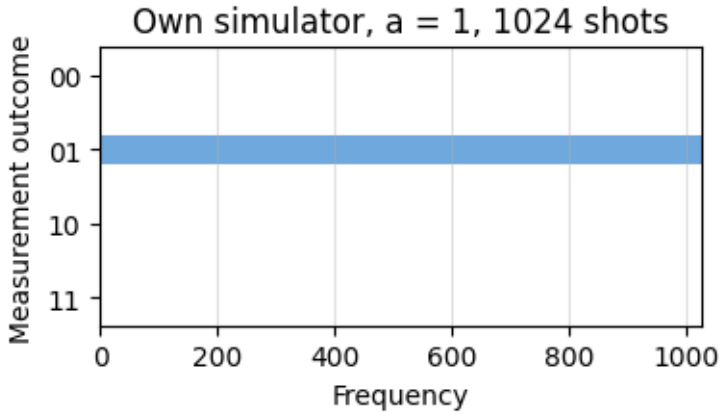
```
                start:  [1. 0. 0. 0.]
            H (x) H:  [0.5 0.5 0.5 0.5]
          oracle: X's:  [0.5 0.5 0.5 0.5]
 oracle: CZ - H on q0:  [0.7071 0.     0.7071 0.    ]
    oracle: CZ - CNOT:  [0.7071 0.     0.     0.7071]
 oracle: CZ - H on q0:  [ 0.5  0.5  0.5 -0.5]
          oracle: X's:  [ 0.5 -0.5  0.5  0.5]
           W: H (x) H:  [ 0.5  0.5 -0.5  0.5]
```

```
           W: Z (x) Z:  [ 0.5 -0.5  0.5  0.5]
      W: CZ - H on q0:  [0.     0.7071 0.7071 0.    ]
         W: CZ - CNOT:  [0.     0.7071 0.     0.7071]
      W: CZ - H on q0:  [ 0.5 -0.5  0.5 -0.5]
           W: H (x) H:  [0. 1. 0. 0.]
Problem 14  --  final state == |01> ? True
```

[18]:
```
# --- Problem 14: measurement statistics, every target a, and a cross-check against Qiskit ---
rng = np.random.default_rng(SEED)
for a in range(4):
    sim = figure1(a)
    cnt = sim.measure(SHOTS, rng)
    qc = QuantumCircuit(2)
    qc.h([0, 1])
    for q in (0, 1):
        if not (a >> q) & 1:
            qc.x(q)
    qc.cz(1, 0)
    for q in (0, 1):
        if not (a >> q) & 1:
            qc.x(q)
    qc.h([0, 1]); qc.z([0, 1]); qc.cz(1, 0); qc.h([0, 1])
    same = np.allclose(Statevector(qc).data, sim.state)
    print(f"Problem 14  --  a = {a} (|{a:02b}>):  counts {cnt},  matches Qiskit Statevector ?␣
 ↪{same}")

fig, ax = plt.subplots(figsize=(4, 2.4))
cnt = figure1(1).measure(SHOTS, rng)
keys = ["11", "10", "01", "00"]
ax.barh(keys, [cnt.get(k, 0) for k in keys], color="#6fa8dc", height=0.4)
ax.set_xlim(0, SHOTS); ax.set_xlabel("Frequency"); ax.set_ylabel("Measurement outcome")
ax.set_title(f"Own simulator, a = 1, {SHOTS} shots"); ax.grid(axis="x", alpha=0.4)
fig.tight_layout()
fig.savefig("../images/problem-14-answer.png", dpi=200, bbox_inches="tight")
display(fig)
plt.close(fig)
```

```
Problem 14  --  a = 0 (|00>):  counts {'00': 1024},  matches Qiskit Statevector
? True
Problem 14  --  a = 1 (|01>):  counts {'01': 1024},  matches Qiskit Statevector
? True
Problem 14  --  a = 2 (|10>):  counts {'10': 1024},  matches Qiskit Statevector
? True
Problem 14  --  a = 3 (|11>):  counts {'11': 1024},  matches Qiskit Statevector
? True
```



1.16 15. Implement Grover's Algorithm with XOR Gate

[19]:
```
# --- Problem 15: XOR oracle for a = 1 with n = 2 query qubits (q2 = MSB, q1) and m = 1 auxiliary␣
  ↪(q0 = LSB) ---
f = lambda x: int(x == 1)
U_xor = xor_oracle(f, 2, 1)                        # |x>_2 |y>_1 -> |x>_2 |y XOR f(x)>_1
for i in range(8):
    j = int(np.argmax(U_xor[:, i]))
    if i != j:
        print(f"Problem 15  --  U_f |{i:03b}> = |{j:03b}>")
display(Math(r"U_f = " + latex_matrix(U_xor)))


def xor_oracle_gates():
    qc = QuantumCircuit(3, name="U_f")
    qc.x(2)                  # x = 01 has q2 = 0: flip it so the Toffoli fires on x = 01
    qc.ccx(2, 1, 0)          # y -> y XOR (q2' AND q1)
    qc.x(2)
    return qc


print("Problem 15  --  X, Toffoli, X matches the matrix ?", np.
  ↪allclose(Operator(xor_oracle_gates()).data, U_xor))
minus = np.array([1, -1]) / np.sqrt(2)
V = np.diag([1, -1, 1, 1])
ok = all(np.allclose(U_xor @ np.kron(ket(format(x, "02b")), minus), np.kron(V @ ket(format(x,␣
  ↪"02b")), minus)) for x in range(4))
print("Problem 15  --  U_f |x>|-> == (V|x>)|-> for every x ?", ok)
```

```
Problem 15  --  U_f |010> = |011>
Problem 15  --  U_f |011> = |010>
```

$$U_f = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

```
Problem 15  --  X, Toffoli, X matches the matrix ? True
Problem 15  --  U_f |x>|-> == (V|x>)|-> for every x ? True
```

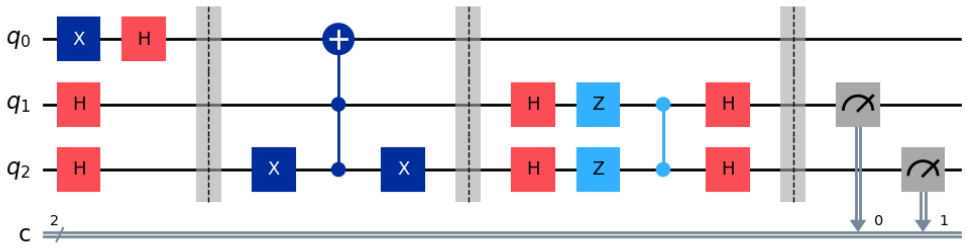
[20]:
```
# --- Problem 15: the circuit of Figure 2 (one WV, since theta = 30 deg) ---
def figure2(iterations=1, uncompute=False):
    q, c = QuantumRegister(3, "q"), ClassicalRegister(3 if uncompute else 2, "c")
    qc = QuantumCircuit(q, c)
    qc.x(0); qc.h(0)                          # auxiliary |0> -> |->
    qc.h([1, 2])                              # |Phi>_2
    for _ in range(iterations):
        qc.barrier()
        qc.compose(xor_oracle_gates(), inplace=True)     # U_f = V
        qc.barrier()
        qc.h([1, 2]); qc.z([1, 2]); qc.cz(2, 1); qc.h([1, 2])   # W
    qc.barrier()
    if uncompute:
```

```
        qc.h(0); qc.x(0)                    # |-> -> |1> -> |0>
        qc.measure([0, 1, 2], [0, 1, 2])    # bit string q2 q1 q0
    else:
        qc.measure([1, 2], [0, 1])          # bit string q2 q1 = x
    return qc


display(figure2().draw("mpl", fold=-1))
plt.close("all")

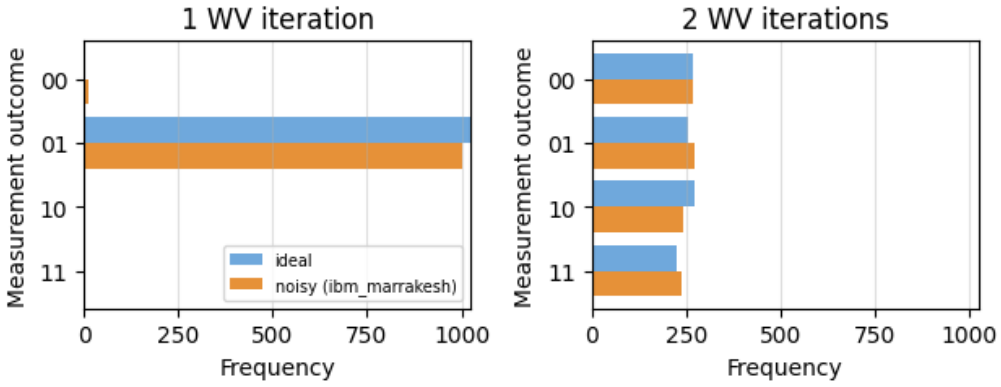
body = figure2().remove_final_measurements(inplace=False)
sv = Statevector(body).data
print("Problem 15  --  final state == |01>|-> ?", np.allclose(sv, np.kron(ket("01"), minus)))

panels = []
for k in (1, 2):
    ci, cn = run(figure2(k), ideal), run(figure2(k), noisy)
    report(ci, f"{k} iteration(s), ideal  (bit string q2 q1)", 2)
    report(cn, f"{k} iteration(s), noisy  (bit string q2 q1)", 2)
    panels.append((f"{k} WV iteration" + ("s" if k > 1 else ""), ci, cn))
ci, cn = run(figure2(1, True), ideal), run(figure2(1, True), noisy)
report(ci, "1 iteration + auxiliary uncomputed, ideal  (q2 q1 q0)", 3)
report(cn, "1 iteration + auxiliary uncomputed, noisy  (q2 q1 q0)", 3)
histograms(panels, "problem-15-answer.png", 2)
tqc = transpile(figure2(), backend, optimization_level=1, seed_transpiler=SEED)
print("Problem 15  --  transpiled for ibm_marrakesh: depth", tqc.depth(), " two-qubit gates", tqc.
  ↪count_ops().get("cz", 0))
```




```
Problem 15  --  final state == |01>|-> ? True
1 iteration(s), ideal  (bit string q2 q1)
  00:     0 / 1024
  01:  1024 / 1024
  10:     0 / 1024
  11:     0 / 1024
1 iteration(s), noisy  (bit string q2 q1)
  00:    13 / 1024
  01:  1003 / 1024
  10:     5 / 1024
  11:     3 / 1024
2 iteration(s), ideal  (bit string q2 q1)
  00:   267 / 1024
  01:   257 / 1024
  10:   275 / 1024
  11:   225 / 1024
2 iteration(s), noisy  (bit string q2 q1)
  00:   267 / 1024
  01:   274 / 1024
```

```
  10:   243 / 1024
  11:   240 / 1024
1 iteration + auxiliary uncomputed, ideal  (q2 q1 q0)
  000:     0 / 1024
  001:     0 / 1024
  010:  1024 / 1024
  011:     0 / 1024
  100:     0 / 1024
  101:     0 / 1024
  110:     0 / 1024
  111:     0 / 1024
1 iteration + auxiliary uncomputed, noisy  (q2 q1 q0)
  000:    11 / 1024
  001:     4 / 1024
  010:   987 / 1024
  011:    10 / 1024
  100:     6 / 1024
  101:     1 / 1024
  110:     5 / 1024
  111:     0 / 1024
```




```
Problem 15  --  transpiled for ibm_marrakesh: depth 50  two-qubit gates 10
```